

Explain me simply ansible variables as there are confusion for me around normal variables, env variables vars in files and in default files and hostvars, groupvars and many with example/syntax



Absolutely. The main reason Ansible variables feel confusing is that **“variable” is one concept, but Ansible gives you many places to define it.**

Think of it like this:

Variable = a value stored somewhere so your playbook doesn't hard-code it.

The different names (vars , group_vars , host_vars , defaults , hostvars , ansible_env) mostly describe **where the value comes from or how you access it.**

1. First understand the basic variable

Instead of:

```
yaml
- name: Install nginx
  apt:
    name: nginx
```

you can do:

```
yaml
vars:
  package_name: nginx

tasks:
- name: Install package
  apt:
    name: "{{ package_name }}"
```

Here:

```
text
```

```
package_name = variable
nginx         = value
{{ package_name }} = use the variable
```

2. vars – variables directly inside a play

This is the simplest form.

yaml

```
---
- name: Web server
  hosts: web
  vars:
    package_name: nginx
    app_port: 8080

  tasks:
    - name: Install package
      apt:
        name: "{{ package_name }}"

    - name: Show port
      debug:
        msg: "Application runs on {{ app_port }}"
```

Think:

text

```
playbook
  ↓
vars
  ↓
tasks use them
```

When to use?

Good for **small/simple values specific to that play.**

3. Variables in a separate vars file

Instead of putting everything inside the playbook:

yaml

```
vars:
  package_name: nginx
  app_port: 8080
```

you can create:

text

```
vars/
└── web.yml
```

vars/web.yml

yaml

```
package_name: nginx
app_port: 8080
environment: production
```

Then:

yaml

```
---
- name: Web server
  hosts: web

  vars_files:
    - vars/web.yml

  tasks:
    - debug:
        msg: "Installing {{ package_name }}"

    - debug:
        msg: "Environment = {{ environment }}"
```

So:

text

```
vars_files
  ↓
vars/web.yml
  ↓
variables become available
  ↓
```

```
tasks
```

Why?

Useful when you have **many variables** and want to keep the playbook clean.

4. `group_vars` — variables for a group of servers

This is extremely important in real Ansible.

Suppose inventory:

```
ini

[web]
web01
web02

[db]
db01
db02
```

You might want:

```
text

web servers → port 8080
db servers  → port 5432
```

Create:

```
text

group_vars/
├── web.yml
└── db.yml
```

`group_vars/web.yml`

```
yaml

app_port: 8080
app_name: nginx
```

`group_vars/db.yml`

```
yaml
```

```
db_port: 5432
db_name: myapp
```

Now:

```
yaml
- hosts: web
  tasks:
    - debug:
        msg: "{{ app_name }}" runs on {{ app_port }}
```

Ansible automatically loads:

```
text

group_vars/web.yml
```

because the server belongs to the `web` group.

5. `host_vars` — variables for ONE specific server

Suppose:

```
ini

[web]
web01
web02
```

But `web01` needs a different port.

Create:

```
text

host_vars/
├── web01.yml
└── web02.yml
```

`host_vars/web01.yml`

```
yaml

app_port: 9000
server_role: primary
```

host_vars/web02.yml

yaml

```
app_port: 8080
server_role: secondary
```

Playbook:

yaml

```
- hosts: web
  tasks:
    - debug:
        msg: "{{ inventory_hostname }}" uses port {{ app_port }}
```

Result:

text

```
web01 → 9000
web02 → 8080
```

So remember:

text

```
group_vars
  ↓
many servers

host_vars
  ↓
one specific server
```

6. The easiest way to remember group_vars vs host_vars

Imagine:

text

```
      web
     /  \
    /      \
web01    web02
```

group_vars/web.yml

Applies to:

```
text
```

```
web01
```

```
web02
```

host_vars/web01.yml

Applies only to:

```
text
```

```
web01
```

That's it.

7. defaults/main.yml – mainly used with Roles

This one confuses many people.

Suppose you create a role:

```
text
```

```
roles/  
├── nginx/  
│   ├── defaults/  
│   │   └── main.yml  
│   ├── tasks/  
│   │   └── main.yml  
│   └── templates/  
└──
```

defaults/main.yml

```
yaml
```

```
nginx_port: 80  
nginx_package: nginx
```

Then:

```
yaml
```

```
# tasks/main.yml  
  
- name: Install nginx  
  apt:
```

```
name: "{{ nginx_package }}"
```

```
- name: Show port
  debug:
    msg: "Nginx port = {{ nginx_port }}"
```

Why defaults ?

Because these are **default values**.

You can override them.

For example:

```
yaml
```

```
vars:
  nginx_port: 8080
```

The role's default:

```
yaml
```

```
nginx_port: 80
```

can be replaced by:

```
yaml
```

```
nginx_port: 8080
```

Think:

defaults = "If nobody gives me a value, I'll use this."

8. vars/main.yml inside a Role

Roles can also have:

```
text
```

```
roles/
├── nginx/
│   ├── defaults/
│   │   └── main.yml
│   ├── vars/
│   │   └── main.yml
│   └── tasks/
```


└── main.yml

roles/nginx/vars/main.yml :

yaml

```
nginx_package: nginx
nginx_service: nginx
```

These are role variables with **higher precedence than defaults**.

Simple mental model:

text

```
defaults/main.yml
  ↓
weak
  ↓
vars/main.yml
  ↓
stronger
```

Don't over-focus on the exact precedence hierarchy initially. Just remember:

defaults = intended to be overridden

role vars = generally more fixed/internal values

9. **hostvars** — this is NOT where you define variables

This is an important distinction.

You define:

text

```
host_vars/
group_vars/
vars/
```

But **hostvars** is something you **use to access variables belonging to another host**.

Suppose:

ini

```
[web]
web01
web02
```

```
[db]
db01
```

host_vars/db01.yml :

```
yaml
```

```
db_private_ip: 10.0.2.50
```

Now you're configuring web01 .

You can access db01 's variable:

```
yaml
```

```
- name: Show DB IP
  debug:
    msg: "{{ hostvars['db01']['db_private_ip'] }}"
```

Output:

```
text
```

```
10.0.2.50
```

Think:

```
text
```

```
hostvars
  ↓
"Give me variables belonging to another host"
```

10. groups — another important built-in variable

Ansible automatically provides inventory information.

Given:

```
ini
```

```
[web]
web01
web02

[db]
db01
```

```
db02
```

You can do:

```
yaml
```

```
- debug:
  msg: "{{ groups['web'] }}"
```

Result:

```
text
```

```
["web01", "web02"]
```

And:

```
yaml
```

```
- debug:
  msg: "{{ groups['db'] }}"
```

Result:

```
text
```

```
["db01", "db02"]
```

You can even do:

```
yaml
```

```
- debug:
  msg: "{{ groups['web'][0] }}"
```

Result:

```
text
```

```
web01
```

Think:

```
text
```

```
groups
```

```
↓
```

```
"Give me hosts belonging to a group"
```

11. inventory_hostname

Another automatically available variable:

```
yaml
- debug:
  msg: "I am {{ inventory_hostname }}"
```

For web01 :

```
text
I am web01
```

For web02 :

```
text
I am web02
```

Very useful when you want to know:

```
"Which server am I currently configuring?"
```

12. Environment variables – different concept

This is where people commonly get confused.

A Linux machine might have:

```
bash
export APP_ENV=production
export DB_HOST=10.0.2.50
```

These are **Linux environment variables**, not normal Ansible variables.

You can check:

```
bash
echo $APP_ENV
```

Ansible can read them using:

```
yaml
```

```
{{ ansible_env.APP_ENV }}
```

Example:

```
yaml
```

```
- name: Show environment variable
  debug:
    msg: "{{ ansible_env.APP_ENV }}"
```

If the environment contains:

```
bash
```

```
APP_ENV=production
```

you get:

```
text
```

```
production
```

13. `ansible_env` vs normal Ansible variable

This distinction is very important.

Normal Ansible variable

```
yaml
```

```
vars:
  app_port: 8080
```

Use:

```
yaml
```

```
{{ app_port }}
```

Linux environment variable

```
bash
```

```
export APP_PORT=8080
```

Use:

```
yaml
```

```
{{ ansible_env.APP_PORT }}
```

So:

```
text
```

Ansible variable



app_port



```
{{ app_port }}
```

Linux environment variable



APP_PORT



```
{{ ansible_env.APP_PORT }}
```

14. You can also SET environment variables for a task

This is different again.

Suppose your application needs:

```
text
```

```
APP_ENV=production
```

You can do:

```
yaml
```

```
- name: Run application
  command: ./app
  environment:
    APP_ENV: production
```

Now:

```
text
```

Ansible



```
starts command
↓
command receives
APP_ENV=production
```

This environment variable exists for that process/task.

15. **set_fact** — create a variable during execution

You can create variables dynamically.

```
yaml

- name: Set application name
  set_fact:
    app_name: nginx
```

Then:

```
yaml

- debug:
    msg: "{{ app_name }}"
```

Output:

```
text

nginx
```

You can also calculate:

```
yaml

- set_fact:
    app_url: "http://{{ inventory_hostname }}:8080"
```

Then:

```
yaml

{{ app_url }}
```

Think:

set_fact = create/change a variable while the playbook is running.

16. Registered variables

Another common one:

```
yaml
- name: Check disk
  command: df -h
  register: disk_result
```

Ansible stores the command result in:

```
text

disk_result
```

Then:

```
yaml
- debug:
    msg: "{{ disk_result.stdout }}"
```

Think:

```
text

command
  ↓
result
  ↓
register
  ↓
variable
```

This is a **registered variable**.

17. Facts — variables Ansible discovers

When Ansible runs:

```
yaml

gather_facts: true
```

it collects information about the server.

For example:

yaml

```
{{ ansible_hostname }}
{{ ansible_distribution }}
{{ ansible_os_family }}
{{ ansible_memory_mb }}
{{ ansible_default_ipv4.address }}
```

Example:

yaml

```
- debug:
  msg: "OS = {{ ansible_distribution }}"
```

Could produce:

text

OS = Ubuntu

These are called **facts**.

Think:

text

```
Ansible connects to server
    ↓
discovers information
    ↓
stores it as variables/facts
```

18. Now put everything together

Imagine this project:

text

```
ansible/
|
|—— inventory
|
|—— playbook.yml
|
|—— group_vars/
|    |—— web.yml
```

```
|   └── db.yml
|
|── host_vars/
|   └── web01.yml
|
|── vars/
|   └── common.yml
|
└── roles/
    └── nginx/
        ├── defaults/
        │   └── main.yml
        ├── vars/
        │   └── main.yml
        └── tasks/
            └── main.yml
```

You might have:

group_vars/web.yml

yaml

```
app_port: 8080
app_name: nginx
```

host_vars/web01.yml

yaml

```
server_priority: primary
```

vars/common.yml

yaml

```
company_name: ABC
```

Role defaults/main.yml

yaml

```
nginx_port: 80
```

Playbook

yaml

```
- name: Configure web servers
  hosts: web

vars_files:
  - vars/common.yml

tasks:

  - debug:
    msg: "Server = {{ inventory_hostname }}"

  - debug:
    msg: "Application = {{ app_name }}"

  - debug:
    msg: "Port = {{ app_port }}"

  - debug:
    msg: "Company = {{ company_name }}"

  - debug:
    msg: "Priority = {{ server_priority }}"

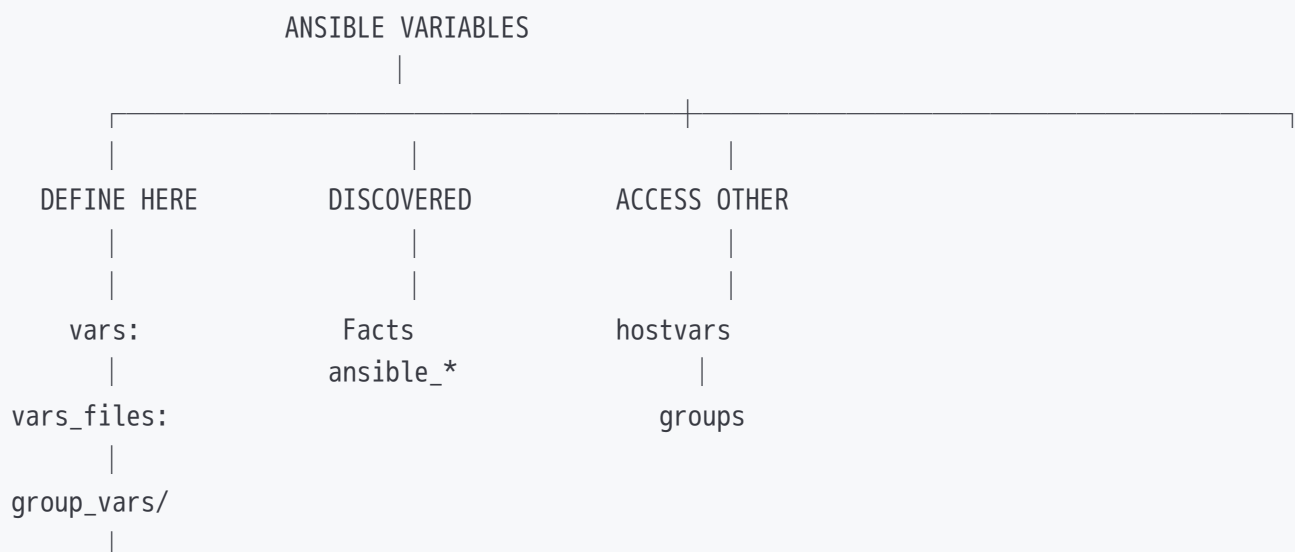
  - debug:
    msg: "OS = {{ ansible_distribution }}"
```

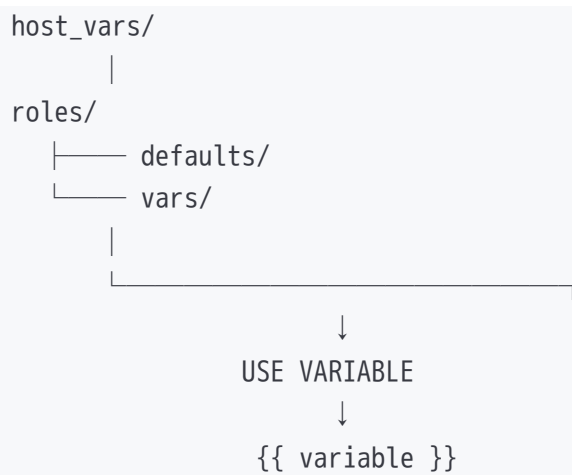
Ansible brings variables from different places together.

19. The mental map you should memorize

Don't memorize 100 precedence rules yet. Remember this:

text





20. Production mental model

This is probably the most useful way for you to think about it.

Imagine you have:

```
text
```

```
100 web servers
10 database servers
```

You don't want to put everything in one playbook.

Common to everyone

```
text
```

```
group_vars/all.yml
```

Example:

```
yaml
```

```
company: ABC
timezone: Asia/Kolkata
```

Common to web servers

```
text
```

```
group_vars/web.yml
```

```
yaml
```

```
app_port: 8080
```

Common to database servers

```
text
```

```
group_vars/db.yml
```

```
yaml
```

```
db_port: 5432
```

Special configuration for one server

```
text
```

```
host_vars/web01.yml
```

```
yaml
```

```
app_port: 9090
```

Role's configurable defaults

```
text
```

```
roles/nginx/defaults/main.yml
```

```
yaml
```

```
nginx_port: 80
```

Role's internal/fixed variables

```
text
```

```
roles/nginx/vars/main.yml
```

```
yaml
```

```
nginx_service: nginx
```

Runtime-generated value

```
yaml
```

```
set_fact:
```

Command output

```
yaml
```

```
register:
```

Information discovered from machine

```
yaml
```

```
ansible_distribution
ansible_hostname
...
```

Linux environment

```
yaml
```

```
ansible_env.PATH
ansible_env.HOME
...
```

Another server's variables

```
yaml
```

```
hostvars['db01']['db_private_ip']
```

21. One final cheat sheet

Thing	Meaning	Example
vars:	Define variable in play	app_port: 8080
vars_files:	Load variables from file	vars_files: vars.yml
group_vars/	Variables for group	group_vars/web.yml
host_vars/	Variables for specific host	host_vars/web01.yml
defaults/main.yml	Role default values	nginx_port: 80

Thing	Meaning	Example
vars/main.yml	Role variables	nginx_service: nginx
set_fact	Create variable during execution	set_fact: x: 10
register	Store task result	register: result
Facts	Server information discovered by Ansible	ansible_hostname
ansible_env	Environment variables from target	ansible_env.PATH
environment:	Set env variable for a task	APP_ENV: prod
hostvars	Access another host's variables	hostvars['db01']['ip']
groups	Access hosts in inventory groups	groups['web']

The 5 things I would focus on first

For your Ansible learning, **don't try to master every variable type at once.**

Learn in this order:

text

1. vars
 - ↓
2. group_vars
 - ↓
3. host_vars
 - ↓
4. defaults/main.yml
 - ↓
5. hostvars

Then learn:

text

```

set_fact
register
facts
ansible_env
environment
variable precedence

```

The key distinction to permanently remember is:

group_vars and **host_vars** are places where you STORE variables.

hostvars is a way to ACCESS variables belonging to another host.

ansible_env is a way to ACCESS environment variables.

defaults provides default values for roles.

vars simply defines variables.

That distinction removes most of the confusion.